

An Analysis of Sequence to sequence learning

Sequence to sequence learning

Sequence to sequence learning -- Part 1

The encoder is responsible for processing the input sequence and capturing its essential information, which is stored as the hidden state of the network and, in a model with attention mechanism, a context vector. The context vector is the weighted sum of the input hidden states and is generated for every time instance in the output sequences.

Sequence to sequence learning -- Part 2

Other applications In 2019, Facebook announced its use in symbolic integration and resolution of differential equations. The company claimed that it could solve complex equations more rapidly and with greater accuracy than commercial solutions such as Mathematica, MATLAB and Maple. First, the equation is parsed into a tree structure to avoid notational idiosyncrasies. An LSTM neural network then applies its standard pattern recognition facilities to process the tree. In 2020, Google released Meena, a 2.6 billion parameter seq2seq-based chatbot trained on a 341 GB data set. Google claimed that the chatbot has 1.7 times greater model capacity than OpenAI's GPT-2. In 2022, Amazon introduced AlexaTM 20B, a moderate-sized (20 billion parameter) seq2seq language model. It uses an encoder-decoder to accomplish few-shot learning. The encoder outputs a representation of the input that the decoder uses as input to perform a specific task, such as translating the input into another language. The model outperforms the much larger GPT-3 in language translation and summarization. Training mixes denoising (appropriately inserting missing text in strings) and causal-language-modeling (meaningfully extending an input text). It allows adding features across different languages without massive training workflows. AlexaTM 20B achieved state-of-the-art performance in few-shot-learning tasks across all Flores-101 language pairs, outperforming GPT-3 on several tasks.

Sequence to sequence learning -- Part 3

This is the dot-attention mechanism. The particular version described in this section is "decoder cross-attention", as the output context vector is used by the decoder, and the input keys and values come from the encoder, but the query comes from the decoder, thus "cross-attention".

Sequence to sequence learning -- Part 4

where $v_0 = h_0 W_V$, $v_1 = h_1 W_V$, ... $\{v_{\{0\}} = h_{\{0\}} W^{\{V\}}, v_{\{1\}} = h_{\{1\}} W^{\{V\}}, \dots\}$ are the value vectors, linearly transformed by another matrix to provide the model with freedom to find the best way to represent values.

Without the matrices W_Q , W_K , W_V $\{W^{\{Q\}}, W^{\{K\}}, W^{\{V\}}\}$, the model would be forced to use the same hidden vector for both key and value, which might not be appropriate, as these two tasks are not the same.

Sequence to sequence learning -- Part 5

As hand-crafting weights defeats the purpose of machine learning, the model must compute the attention weights on its own. Taking analogy from the language of database queries, we make the model construct a triple of vectors: key, query, and value. The rough idea is that we have a "database" in the form of a list of key-value pairs. The decoder sends in a query, and obtains a reply in the form of a weighted sum of the values, where the weight is proportional to how closely the query resembles each key. The decoder first processes the "<start>" input partially, to obtain an intermediate vector h_0^d $\{h_{\{0\}}^d\}$, the 0th hidden vector of decoder. Then, the intermediate vector is transformed by a linear map W_Q $\{W^{\{Q\}}\}$ into a query vector $q_0 = h_0^d W_Q$ $\{q_{\{0\}} = h_{\{0\}}^d W^{\{Q\}}\}$. Meanwhile, the hidden vectors outputted by the encoder are transformed by another linear map W_K $\{W^{\{K\}}\}$ into key vectors $k_0 = h_0 W_K$, $k_1 = h_1 W_K$, ... $\{k_{\{0\}} = h_{\{0\}} W^{\{K\}}, k_{\{1\}} = h_{\{1\}} W^{\{K\}}, \dots\}$. The linear maps are useful for providing the model with enough freedom to find the best way to represent the data. Now, the query and keys are compared by taking dot products: $q_0 k_0^T$, $q_0 k_1^T$, ... $\{q_{\{0\}} k_{\{0\}}^T, q_{\{0\}} k_{\{1\}}^T, \dots\}$. Ideally, the model should have learned to compute the keys and values, such that $q_0 k_0^T$ $\{q_{\{0\}} k_{\{0\}}^T\}$ is large, $q_0 k_1^T$ $\{q_{\{0\}} k_{\{1\}}^T\}$ is small, and the rest are very small. This can be interpreted as saying that the attention weight should be mostly applied to the 0th hidden vector of the encoder, a little to the 1st, and essentially none to the rest. In order to make a properly weighted sum, we need to transform this list of dot products into a probability distribution over 0, 1, ... $\{0, 1, \dots\}$. This can be accomplished by the softmax function, thus giving us the attention weights: $(w_{00}, w_{01}, \dots) = \text{softmax}(q_0 k_0^T, q_0 k_1^T, \dots)$ $\{w_{\{00\}}, w_{\{01\}}, \dots\} = \text{softmax}\{q_{\{0\}} k_{\{0\}}^T, q_{\{0\}} k_{\{1\}}^T, \dots\}$. This is then used to compute the context vector: $c_0 = w_{00} v_0 + w_{01} v_1 + \dots$ $\{c_{\{0\}} = w_{\{00\}} v_{\{0\}} + w_{\{01\}} v_{\{1\}} + \dots\}$.

Sequence to sequence learning -- Part 6

$(h_0, h_1, \dots)$ $\{h_{\{0\}}, h_{\{1\}}, \dots\}$, "<start>" $\rightarrow$ "la" $(h_0, h_1, \dots)$ $\{h_{\{0\}}, h_{\{1\}}, \dots\}$, "<start>"

An Analysis of Sequence to sequence learning

Sequence to sequence learning

la") $\rightarrow$ "la zone" ($h_0, h_1, \dots$) $\rightarrow$ "la zone de" ($h_0, h_1, \dots$) $\rightarrow$ "la zone de contrôle international" $\rightarrow$ "la zone de contrôle international <end>" Here, we use the special <start> token as a control character to delimit the start of input for the decoder. The decoding terminates as soon as "<end>" appears in the decoder output.

Sequence to sequence learning -- Part 7

History One naturally wonders if the problem of translation could conceivably be treated as a problem in cryptography. When I look at an article in Russian, I say: 'This is really written in English, but it has been coded in some strange symbols. I will now proceed to decode.'

Sequence to sequence learning -- Part 8

The decoder takes the context vector and hidden states from the encoder and generates the final output sequence. The decoder operates in an autoregressive manner, producing one element of the output sequence at a time. At each step, it considers the previously generated elements, the context vector, and the input sequence information to make predictions for the next element in the output sequence. Specifically, in a model with attention mechanism, the context vector and the hidden state are concatenated together to form an attention hidden vector, which is used as an input for the decoder. The seq2seq method developed in the early 2010s uses two neural networks: an encoder network converts an input sentence into numerical vectors, and a decoder network converts those vectors to sentences in the target language. The Attention mechanism was grafted onto this structure in 2014 and is shown below. Later it was refined into the encoder-decoder Transformer architecture of 2017.